Benchmarking Matrix Multiplication on GPUs

Tripatjit Kaur

August 2026

1 Introduction

Matrices lie at the core of current deep learning algorithms. While training, these models require multiple matrix multiplications which can go up to millions of parameters in size. Therefore, more efficient matrix multiplication algorithms are essential to speed up the training period for these models. However, the naive matrix multiplication method, i.e., $C_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}$ has a time complexity of $O(n^3)$, which is incredibly slow at a large scale.

While multiple faster matrix multiplication algorithms have been suggested theoretically, none of them have proven to be efficient when compared to the naive method on GPUs. These sub-cubic algorithms work on a divide-and-conquer principle, i.e., they recursively split the matrices into smaller submatrices to get the matrix product. Thus, these algorithms incur huge memory access costs, and they suffer from more floating point errors than the naive method due to extra additions. Moreover, some such algorithms are only faster when the matrix size becomes incredibly large, hence earning the name of "galactic algorithms". Our primary focus is to reduce the communication costs which slow down these galactic algorithms on GPUs.

Therefore, this project aims to find and test different alterations to these sub-cubic algorithms which can allow them to run at a speed comparable to the current naive algorithm. In this project, we looked at three algorithms- Strassen (recursive), Strassen (iterative), and Coppersmith-Winograd.

1.1 Recursive Strassen Algorithm

Strassen algorithm has a time complexity of $O(n^{log_2 7})$ [3]. The algorithm recursively splits the matrix into four equally-sized sub-matrices [3]. As a result, the number of multiplications go from 8 to 7 [2]. However, number of additions goes up from 4 to 18 [2]. The formulation used below follows Introduction to Algorithms by Cormen et al. [3]. For our implementation of recursive Strassen algorithm, please refer to the "strassen.py" file on GitHub.

1.2 Iterative Strassen Algorithm

As recursive algorithms are not optimized in python, we implemented an iterative version. To do so, we implemented a stack which kept track of different matrices and iterated through each stack position. We adapted the stack structure from Zeil's notes on "Converting Recursion to Iteration"

[4]. The stack structure is as follows:

$$\mathcal{S} = \Big[\big(A,\ B,\ \underbrace{\perp, \ldots, \perp}_{P_1,\ldots,P_7},\ \underbrace{\perp, \perp, \perp, \perp}_{C_{11},C_{12},C_{21},C_{22}}\ ,\ 1\big) \Big]$$

.

The iterative implementation has the same time complexity as the recursive Strassen algorithm. To learn more about our iteration implementation, please refer to "strassen_iter.py" on GitHub.

1.3 Coppersmith-Winograd Algorithm

Like Strassen, Coppersmith-Winograd is also a divide-and-conquer algorithm. However, while Strassen had a time complexity of $O(n^{log_2 7})$, Coppersmith-Winograd further reduces it to $O(n^{2.376})$ [2]. It does so by keeping the same number of matrix multiplications while reducing the additions to 15 as compared to Strassen's 18 [2]. However, Winograd requires matrix sizes to be gigantic to offer a considerable boost, hence earning the name "galactic algorithm". Our implementation of the algorithm is based on Aho, Hopcroft, and Ullman's The Design and Analysis of Computer Algorithms [1]. You can find our implementation in "strassen_winograd.py" (which also includes an iterative version).

2 Methods

We compared the divide-and-conquer algorithms with the native matmul function in PyTorch and Julia. In addition to comparing pure Strassen vs. Naive method, we also compared hybrid Strassen methods, where the multiplication algorithm would follow the native matmul algorithm up until a certain matrix size, N and switch to Strassen after hitting the threshold. The matrix sizes ranged from 512 to 16,384, with thresholds varying from 64 to 256. Also, The functions were compared on Nvidia H100 GPUs, with each run using one-eighth of the GPU. Please refer to the "strassen_rec_time.py" file to learn more about the structure of our test runs.

3 Results

Our first test was a comparison between recursive Strassen, iterative Strassen, and the native matmul method. Surprisingly, the iterative version performed the worst and the gap between the three methods continues to increase as the matrix sizes increase. Please refer to fig. 1 below.

Our next test was focused on performance of hybrid Strassen vs. native matmul on Julia. Here, the threshold was N=128. Here, the performance again worsened as the matrix sizes increased. Please refer to fig. 2 below.

Therefore, after multiple tests, we were unable to beat the native matmul. However, we consistently had faster times for Strassen only under one condition- when the matrices had size 128 with a threshold of 64 by a 13% margin.

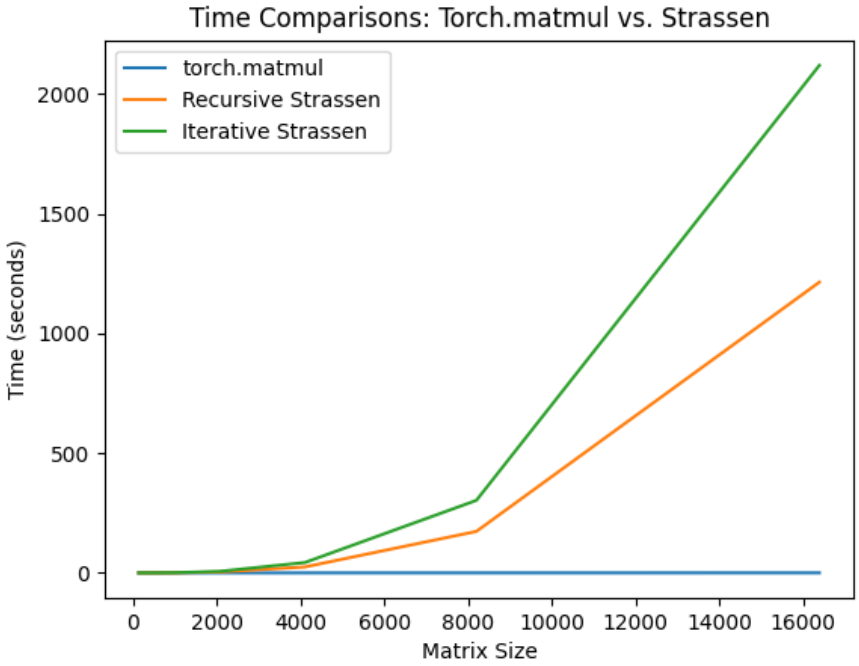


Figure 1: Running time of the matrix multiplication implementations (PyTorch)

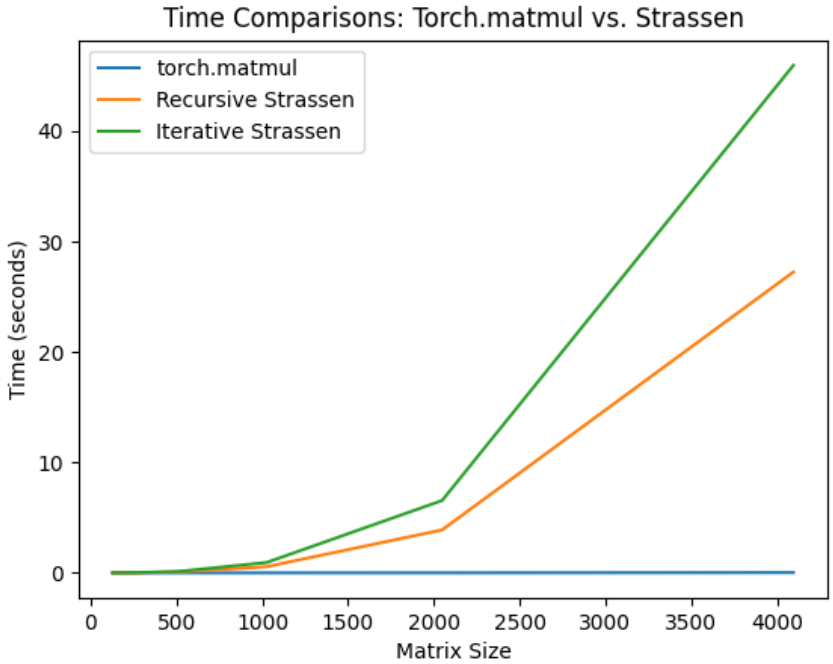


Figure 2: Running time of the matrix multiplication implementations (Julia)

4 Improvements

While we were unable to beat the CuBLAS-optimized matmul function, we found certain methods that can reduce the gap between the different algorithms, such as hybrid methods, pre-allocation, and batching.

One reason for our algorithms' worsened performance was the time allotted for CPU-GPU communication. While the native function stayed on the GPU and only had to relay the results once, the self-implemented functions had to spend a considerable amount of time sharing information from the CPU to the GPU. As a result, the faster computation times came at the cost of much higher communication times, which worsened the performance as the size increased. Therefore, strategies that minimize communication between CPUs and GPUs could greatly help reduce performance times for these algorithms.

4.1 Hybrid Methods

As discussed above, sub-cubic algorithms need matrix sizes to be huge in order to perform optimally. As using the native function would reduce recursive calls for smaller sub-matrices, we can modify the function to only use the divide-and-conquer algorithms for matrices beyond a specific size. Therefore, one technique that helped during our testing period was using methods that switched between native matmul and our self-implemented functions. Additionally, we were only able to outperform the pre-written matmul function when the threshold was 50% of the size of the matrix to be computed.

4.2 Pre-Allocation

Pre-allocation refers to allotting memory space for certain objects before the program starts to run. As GPUs need to communicate with CPUs to allocate space for matrices, having some memory already assigned for the matrix would prevent additional calls to the CPU. As a result, we would be able to save on some of the back-and-forth between the two, and further reduce our implementation's performance.

4.3 Batching

As the name suggests, batching refers to combining multiple operations into a single operation which can be processed at the same time. In our case, we have multiple recursive calls which need to be processed one at a time. Thus, batching would allow all seven multiplication calls to go through to the GPU at the same time. Hence, the communication costs for seven separate calls will be replaced by just one call, which can drastically reduce the running time for matrices with thousands of elements.

5 Conclusion

Pre-written matmul functions have been optimized for matrix multiplication. However, as computational demands increase due to AI, we need faster methods for more streamlined training processes. Therefore, focusing on improving sub-cubic algorithms' performance on GPUs can greatly support future models' implementation.

We tried to modify pre-existing sub-cubic algorithms in order to improve their performance on GPUs. While we were unable to do so, we found the different conditions under which they can perform similar to the matmul function on PyTorch and Julia. Further research into optimal thresholds, and batching and pre-allocation techniques is needed to conclude the level of difference these strategies can make.

References

[1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, Reading, MA, 1974.

[2] Murat Cenk and M. Anwar Hasan. On the arithmetic complexity of strassen-like matrix multiplications. *Journal of Symbolic Computation*, 80:484–501, 2017.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009.

[4] Steven J. Zeil. Converting recursion to iteration. `https://www.cs.odu.edu/~zeil/cs361/latest/Public/recursionConversion/index.html`. Section: Conversion Using Stacks. Accessed: 2026-10-12.